

```
#!/usr/bin/env python3
"""
chess-server.py – real online matchmaking +
move relay for LiveChess.
```

This is what makes "Play Online" on `index_spruced-1-1.html` actually connect two different people in two different places: it pairs up whoever taps "Find an Opponent" next, tells each of them their color, and relays only their opponent's moves back and forth. Nothing about the game itself (legality, checkmate, etc.) happens here – that's still `chess.js` running in each browser – this server only pairs people up and passes moves between exactly two sockets.

There are no accounts yet, so every opponent is shown as "Unknown".

Install (Termux or any Linux):
 `pip install websockets --break-system-packages`

Run:
 `python chess-server.py`

Then, on the LiveChess page, set (before the page's own scripts run):

```
<script>window.ONLINE_WS_URL =  
"ws://<this-machine's-address>:8765";  
</script>
```

so the "Play Online" button knows where to connect. On the same device/network you can leave it as the default (ws://<page-host>:8765); for two different networks this machine needs a public address (a cloud VM, a tunnel like ngrok/Cloudflare Tunnel, or port-forwarding).
"""

```
import asyncio  
import json  
import logging  
import os  
import random
```

```
import websockets
```

```
logging.basicConfig(level=logging.INFO,  
format="%(asctime)s %(message)s")  
log = logging.getLogger("chess-server")
```

```
WAITING = None  
ROOMS = {}
```

```
async def send(ws, message: dict):  
    try:  
        await ws.send(json.dumps(message))
```

```
except
websockets.exceptions.ConnectionClosed:
    pass

async def find_match(ws):
    global WAITING

    if WAITING is None or WAITING is ws:
        WAITING = ws
        await send(ws, {"type": "status",
"text": "Waiting for opponent..."})
        return

    opponent = WAITING
    WAITING = None
    ROOMS[ws] = opponent
    ROOMS[opponent] = ws

    white, black = random.sample([ws,
opponent], 2)

    await send(white, {
        "type": "matched",
        "color": "w",
        "opponent": "Unknown"
    })

    await send(black, {
        "type": "matched",
        "color": "b",
        "opponent": "Unknown"
    })
```

```
        log.info("matched two players into a
game")

async def relay_move(ws, data: dict):
    opponent = ROOMS.get(ws)

    if not opponent:
        await send(ws, {
            "type": "status",
            "text": "No opponent connected
yet."
        })
        return

    await send(opponent, {
        "type": "move",
        "from": data.get("from"),
        "to": data.get("to"),
        "promotion": data.get("promotion",
"q"),
    })

async def handle_client(ws):
    global WAITING

    log.info("client connected")

    try:
        async for raw in ws:
            try:
                data = json.loads(raw)
            except json.JSONDecodeError:
```

```

        continue

        msg_type = data.get("type")

        if msg_type == "find_match":
            await find_match(ws)

        elif msg_type == "move":
            await relay_move(ws, data)

    except
    websockets.exceptions.ConnectionClosed:
        pass

    finally:
        if WAITING is ws:
            WAITING = None

        opponent = ROOMS.pop(ws, None)

        if opponent:
            ROOMS.pop(opponent, None)
            await send(opponent, {"type":
"opponent_left"})

        log.info("client disconnected")

async def main(host="0.0.0.0", port=None):
    # Cloud hosts (Render, Railway, Fly.io,
    etc.) tell the app which port
    # to listen on via the $PORT
    environment variable. Running locally
    # (no $PORT set) still defaults to

```

```
8765, same as before.
    if port is None:
        port = int(os.environ.get("PORT",
8765))
    async with
websockets.serve(handle_client, host,
port):
        log.info(
            "chess matchmaking server
running on %s:%d",
            host,
            port
        )
        await asyncio.Future()

if __name__ == "__main__":
    asyncio.run(main())
```